

1. Write a Python script using pymongo to connect to MongoDB, handle connection errors, and select the `university_mongo_assignment` database and `students_data` collection.

```
from pymongo import MongoClient
from pymongo.errors import ConnectionFailure

MONGO_URI = "mongodb://localhost:27017/"

try:
    client = MongoClient(
        MONGO_URI,
        serverSelectionTimeoutMS=5000
    )

    client.admin.command("ping")

    print("MongoDB connected successfully!")

except ConnectionFailure as e:
    print("MongoDB connection failed:", e)
    exit()

db = client["university_mongo_assignment"]

students_collection = db["students_data"]

print("Database:", db.name)
print("Collection:", students_collection.name)
```

Output:

```
runfile('D:/TyBsc39/TY/Assign3 Q1.py', wdir='D:/TyBsc39/TY')
MongoDB connected successfully!
Database: university_mongo_assignment
Collection: students_data
```

2. Write a Python script to insert 6–8 student documents into the students_data collection using insert_one() and insert_many(), with varied enrollments and marks including "Pending".

```
from pymongo import MongoClient
from pymongo.errors import ConnectionFailure, DuplicateKeyError
client = MongoClient("mongodb://localhost:27017/")
db = client["university_mongo_assignment"]
students_collection = db["students_data"]
alice = {
    "name": "Alice Smith",
    "email": "alice.s@example.com",
    "major": "Computer Science",
    "enrollments": [
        {
            "course_code": "CS101",
            "title": "Intro to Programming",
            "marks": 85
        },
        {
            "course_code": "MA201",
            "title": "Calculus",
            "marks": 91
        }
    ]
}
try:
    result = students_collection.insert_one(alice)
    print("Alice inserted successfully!")
    print("Inserted ID:", result.inserted_id)
except DuplicateKeyError:
    print("Alice already exists.")
students = [
    {
        "name": "Bob Builder",
        "email": "bob.b@example.com",
        "major": "Computer Science",
        "enrollments": [
            {
                "course_code": "CS101",
                "title": "Intro to Programming",
                "marks": 78
            },
            {
                "course_code": "MA201",
                "title": "Calculus",
                "marks": "Pending"
            }
        ]
    },
    {
        "name": "Charlie Brown",
        "email": "charlie.b@example.com",
```

```
"major": "Physics",
"enrollments": [
  {
    "course_code": "PH203",
    "title": "Quantum Mechanics",
    "marks": 94
  },
  {
    "course_code": "PH204",
    "title": "Thermodynamics",
    "marks": 88
  }
]
},
{
  "name": "Diana Prince",
  "email": "diana.p@example.com",
  "major": "Physics",
  "enrollments": [
    {
      "course_code": "PH203",
      "title": "Quantum Mechanics",
      "marks": 89
    },
    {
      "course_code": "MA201",
      "title": "Calculus",
      "marks": 92
    }
  ]
},
{
  "name": "Ethan Hunt",
  "email": "ethan.h@example.com",
  "major": "Arts",
  "enrollments": []
},
{
  "name": "Fiona Green",
  "email": "fiona.g@example.com",
  "major": "Arts",
  "enrollments": [
    {
      "course_code": "AR101",
      "title": "Art History",
      "marks": 86
    }
  ]
},
{
  "name": "George Wilson",
  "email": "george.w@example.com",
```

```
"major": "Mathematics",
"enrollments": [
    {
        "course_code": "MA201",
        "title": "Calculus",
        "marks": 96
    },
    {
        "course_code": "MA202",
        "title": "Linear Algebra",
        "marks": 93
    }
]
},
{
    "name": "Helen Carter",
    "email": "helen.c@example.com",
    "major": "Computer Science",
    "enrollments": [
        {
            "course_code": "CS101",
            "title": "Intro to Programming",
            "marks": "Pending"
        }
    ]
}
]

try:
    result = students_collection.insert_many(
        students,
        ordered=False
    )
    print("\nStudents inserted successfully!")
    print("Number of students inserted:",
          len(result.inserted_ids))
except Exception as e:
    print("Error during insertion:", e)
```

Output:

```
In [2]: runfile('D:/TyBsc39/TY/Assign3 Q2.py', wdir='D:/TyBsc39/TY')

Output from spyder call 'get_namespace_view':
Alice inserted successfully!
Inserted ID: 6a79a074d928c8afb1d47f80

Students inserted successfully!
Number of students inserted: 7
```

3. Write a Python script to perform basic, filtered, sorted, paginated, and aggregation-based retrieval operations on the students_data collection.

```
from pymongo import MongoClient

client = MongoClient("mongodb://localhost:27017/")

db = client["university_mongo_assignment"]

students_collection = db["students_data"]

print("\n===== 3.1 ALL STUDENTS =====")
for student in students_collection.find():
    print(student)

print("Total number of students:",
      students_collection.count_documents({}))

print("\n===== 3.2 STUDENT BY EMAIL =====")
email = "alice.s@example.com"
print(students_collection.find_one({"email": email}))

print("\n===== 3.3 COMPUTER SCIENCE STUDENTS =====")
for student in students_collection.find({"major": "Computer Science"}):
    print(student)

print("\n===== 3.4 STUDENTS ENROLLED IN CS101 =====")
for student in students_collection.find({"enrollments.course_code": "CS101"}):
    print(student)

print("\n===== 3.5 STUDENTS WITH PENDING MARKS =====")
pipeline = [
    {"$unwind": "$enrollments"},
    {"$match": {"enrollments.marks": "Pending"}},
    {"$project": {
        "_id": 0,
        "name": 1,
        "email": 1,
        "course_code": "$enrollments.course_code",
        "title": "$enrollments.title"
    }}
]

for student in students_collection.aggregate(pipeline):
    print(student)

print("\n===== 3.6 TOP 3 STUDENTS =====")
for student in students_collection.find().sort("name", 1).limit(3):
    print(student)

print("\n===== 3.7 PAGINATION =====")
for student in students_collection.find().sort("name", 1).skip(2).limit(3):
```

```
print(student)

print("\n===== 3.8 STUDENTS PER MAJOR =====")
pipeline = [
    {"$group": {
        "_id": "$major",
        "total_students": {"$sum": 1}
    }},
    {"$sort": {"_id": 1}}
]

for result in students_collection.aggregate(pipeline):
    print("Major:", result["_id"],
        "| Count:", result["total_students"])

print("\n===== 3.9 AVERAGE MARKS FOR PH203 =====")
pipeline = [
    {"$unwind": "$enrollments"},
    {"$match": {
        "enrollments.course_code": "PH203",
        "enrollments.marks": {"$type": "number"}
    }},
    {"$group": {
        "_id": "$enrollments.course_code",
        "average_marks": {"$avg": "$enrollments.marks"}
    }}
]

for result in students_collection.aggregate(pipeline):
    print("Course:", result["_id"])
    print("Average Marks:", result["average_marks"])

print("\n===== 3.10 STUDENTS WITH MARKS ABOVE 90 =====")
pipeline = [
    {"$unwind": "$enrollments"},
    {"$match": {"enrollments.marks": {"$gt": 90}}},
    {"$project": {
        "_id": 0,
        "name": 1,
        "course_code": "$enrollments.course_code",
        "marks": "$enrollments.marks"
    }}
]

for student in students_collection.aggregate(pipeline):
    print(student)

client.close()
```

Output:

```

In [3]: runfile('D:/TyBsc39/TY/untitled3.py', wdir='D:/TyBsc39/TY')

===== 3.1 ALL STUDENTS =====
{'_id': ObjectId('6a79a074d928c8afb1d47f80'), 'name': 'Alice Smith', 'email': 'alice.s@example.com', 'major': 'Computer Science', 'enrollments': [{'course_code': 'CS101', 'title': 'Intro to Programming', 'marks': 85}, {'course_code': 'MA201', 'title': 'Calculus', 'marks': 91}]}
{'_id': ObjectId('6a79a074d928c8afb1d47f81'), 'name': 'Bob Builder', 'email': 'bob.b@example.com', 'major': 'Computer Science', 'enrollments': [{'course_code': 'CS101', 'title': 'Intro to Programming', 'marks': 78}, {'course_code': 'MA201', 'title': 'Calculus', 'marks': 'Pending'}]}
{'_id': ObjectId('6a79a074d928c8afb1d47f82'), 'name': 'Charlie Brown', 'email': 'charlie.b@example.com', 'major': 'Physics', 'enrollments': [{'course_code': 'PH203', 'title': 'Quantum Mechanics', 'marks': 94}, {'course_code': 'PH204', 'title': 'Thermodynamics', 'marks': 88}]}
{'_id': ObjectId('6a79a074d928c8afb1d47f83'), 'name': 'Diana Prince', 'email': 'diana.p@example.com', 'major': 'Physics', 'enrollments': [{'course_code': 'PH203', 'title': 'Quantum Mechanics', 'marks': 89}, {'course_code': 'MA201', 'title': 'Calculus', 'marks': 92}]}
{'_id': ObjectId('6a79a074d928c8afb1d47f84'), 'name': 'Ethan Hunt', 'email': 'ethan.h@example.com', 'major': 'Arts', 'enrollments': []}
{'_id': ObjectId('6a79a074d928c8afb1d47f85'), 'name': 'Fiona Green', 'email': 'fiona.g@example.com', 'major': 'Arts', 'enrollments': [{'course_code': 'AR101', 'title': 'Art History', 'marks': 86}]}
{'_id': ObjectId('6a79a074d928c8afb1d47f86'), 'name': 'George Wilson', 'email': 'george.w@example.com', 'major': 'Mathematics', 'enrollments': [{'course_code': 'MA201', 'title': 'Calculus', 'marks': 96}, {'course_code': 'MA202', 'title': 'Linear Algebra', 'marks': 93}]}
{'_id': ObjectId('6a79a074d928c8afb1d47f87'), 'name': 'Helen Carter', 'email': 'helen.c@example.com', 'major': 'Computer Science', 'enrollments': [{'course_code': 'CS101', 'title': 'Intro to Programming', 'marks': 'Pending'}]}
Total number of students: 8

===== 3.2 STUDENT BY EMAIL =====
{'_id': ObjectId('6a79a074d928c8afb1d47f80'), 'name': 'Alice Smith', 'email': 'alice.s@example.com', 'major': 'Computer Science', 'enrollments': [{'course_code': 'CS101', 'title': 'Intro to Programming', 'marks': 85}, {'course_code': 'MA201', 'title': 'Calculus', 'marks': 91}]}

===== 3.3 COMPUTER SCIENCE STUDENTS =====
{'_id': ObjectId('6a79a074d928c8afb1d47f80'), 'name': 'Alice Smith', 'email': 'alice.s@example.com', 'major': 'Computer Science', 'enrollments': [{'course_code': 'CS101', 'title': 'Intro to Programming', 'marks': 85}, {'course_code': 'MA201', 'title': 'Calculus', 'marks': 91}]}
{'_id': ObjectId('6a79a074d928c8afb1d47f81'), 'name': 'Bob Builder', 'email': 'bob.b@example.com', 'major': 'Computer Science', 'enrollments': [{'course_code': 'CS101', 'title': 'Intro to Programming', 'marks': 78}, {'course_code': 'MA201', 'title': 'Calculus', 'marks': 'Pending'}]}
{'_id': ObjectId('6a79a074d928c8afb1d47f87'), 'name': 'Helen Carter', 'email': 'helen.c@example.com', 'major': 'Computer Science', 'enrollments': [{'course_code': 'CS101', 'title': 'Intro to Programming', 'marks': 'Pending'}]}

===== 3.4 STUDENTS ENROLLED IN CS101 =====
{'_id': ObjectId('6a79a074d928c8afb1d47f80'), 'name': 'Alice Smith', 'email': 'alice.s@example.com', 'major': 'Computer Science', 'enrollments': [{'course_code': 'CS101', 'title': 'Intro to Programming', 'marks': 85}, {'course_code': 'MA201', 'title': 'Calculus', 'marks': 91}]}
{'_id': ObjectId('6a79a074d928c8afb1d47f81'), 'name': 'Bob Builder', 'email': 'bob.b@example.com', 'major': 'Computer Science', 'enrollments': [{'course_code': 'CS101', 'title': 'Intro to Programming', 'marks': 78}, {'course_code': 'MA201', 'title': 'Calculus', 'marks': 'Pending'}]}
{'_id': ObjectId('6a79a074d928c8afb1d47f87'), 'name': 'Helen Carter', 'email': 'helen.c@example.com', 'major': 'Computer Science', 'enrollments': [{'course_code': 'CS101', 'title': 'Intro to Programming', 'marks': 'Pending'}]}

===== 3.5 STUDENTS WITH PENDING MARKS =====
{'name': 'Bob Builder', 'email': 'bob.b@example.com', 'course_code': 'MA201', 'title': 'Calculus'}
{'name': 'Helen Carter', 'email': 'helen.c@example.com', 'course_code': 'CS101', 'title': 'Intro to Programming'}

===== 3.6 TOP 3 STUDENTS =====
{'_id': ObjectId('6a79a074d928c8afb1d47f80'), 'name': 'Alice Smith', 'email': 'alice.s@example.com', 'major': 'Computer Science', 'enrollments': [{'course_code': 'CS101', 'title': 'Intro to Programming', 'marks': 85}, {'course_code': 'MA201', 'title': 'Calculus', 'marks': 91}]}
{'_id': ObjectId('6a79a074d928c8afb1d47f81'), 'name': 'Bob Builder', 'email': 'bob.b@example.com', 'major': 'Computer Science', 'enrollments': [{'course_code': 'CS101', 'title': 'Intro to Programming', 'marks': 78}, {'course_code': 'MA201', 'title': 'Calculus', 'marks': 'Pending'}]}
{'_id': ObjectId('6a79a074d928c8afb1d47f82'), 'name': 'Charlie Brown', 'email': 'charlie.b@example.com', 'major': 'Physics', 'enrollments': [{'course_code': 'PH203', 'title': 'Quantum Mechanics', 'marks': 94}, {'course_code': 'PH204', 'title': 'Thermodynamics', 'marks': 88}]}

===== 3.7 PAGINATION =====
{'_id': ObjectId('6a79a074d928c8afb1d47f82'), 'name': 'Charlie Brown', 'email': 'charlie.b@example.com', 'major': 'Physics', 'enrollments': [{'course_code': 'PH203', 'title': 'Quantum Mechanics', 'marks': 94}, {'course_code': 'PH204', 'title': 'Thermodynamics', 'marks': 88}]}
{'_id': ObjectId('6a79a074d928c8afb1d47f83'), 'name': 'Diana Prince', 'email': 'diana.p@example.com', 'major': 'Physics', 'enrollments': [{'course_code': 'PH203', 'title': 'Quantum Mechanics', 'marks': 89}, {'course_code': 'MA201', 'title': 'Calculus', 'marks': 92}]}
{'_id': ObjectId('6a79a074d928c8afb1d47f84'), 'name': 'Ethan Hunt', 'email': 'ethan.h@example.com', 'major': 'Arts', 'enrollments': []}

===== 3.8 STUDENTS PER MAJOR =====
Major: Arts | Count: 2
Major: Computer Science | Count: 3
Major: Mathematics | Count: 1
Major: Physics | Count: 2

===== 3.9 AVERAGE MARKS FOR PH203 =====
Course: PH203
Average Marks: 91.5

===== 3.10 STUDENTS WITH MARKS ABOVE 90 =====
{'name': 'Alice Smith', 'course_code': 'MA201', 'marks': 91}
{'name': 'Charlie Brown', 'course_code': 'PH203', 'marks': 94}
{'name': 'Diana Prince', 'course_code': 'MA201', 'marks': 92}
{'name': 'George Wilson', 'course_code': 'MA201', 'marks': 96}
{'name': 'George Wilson', 'course_code': 'MA202', 'marks': 93}

```

4. Write a Python script to perform update operations on the students_data collection, including modifying student details, adding/updating/removing enrollments, and performing an insert operation.

```
from pymongo import MongoClient
client = MongoClient("mongodb://localhost:27017/")
db = client["university_mongo_assignment"]
students_collection = db["students_data"]
result = students_collection.update_one(
    {"email": "alice.s@example.com"},
    {
        "$set": {
            "major": "Data Science"
        }
    }
)

print("4.1 - Alice major updated")
print("Modified documents:", result.modified_count)
new_course = {
    "course_code": "CS205",
    "title": "Database Management",
    "marks": 90
}

result = students_collection.update_one(
    {"name": "Alice Smith"},
    {
        "$push": {
            "enrollments": new_course
        }
    }
)
print("\n4.2 - New enrollment added to Alice")
print("Modified documents:", result.modified_count)
result = students_collection.update_one(
    {
        "email": "bob.b@example.com",
        "enrollments.course_code": "MA201"
    },
    {
        "$set": {
            "enrollments.$.marks": 75
        }
    }
)
print("\n4.3 - Bob's MA201 marks updated")
print("Modified documents:", result.modified_count)
result = students_collection.update_one(
    {
        "email": "diana.p@example.com"
    },
    {
```

```
"$pull": {
  "enrollments": {
    "course_code": "PH203"
  }
}
}
)
print("\n4.4 - PH203 enrollment removed from Diana")
print("Modified documents:", result.modified_count)
email = "newstudent@example.com"
result = students_collection.update_one(
  {
    "email": email
  },
  {
    "$set": {
      "name": "New Student",
      "major": "Computer Science",
      "email": email
    },
    "$setOnInsert": {
      "enrollments": []
    }
  },
  upsert=True
)
print("\n4.5 - Upsert operation")
if result.upserted_id:
  print("New student inserted.")
  print("Inserted ID:", result.upserted_id)
else:
  print("Existing student updated.")
  print("Modified count:", result.modified_count)
client.close()
```

Output:

```
In [5]: runfile('D:/TyBsc39/TY/untitled4.py', wdir='D:/TyBsc39/TY')
4.1 - Alice major updated
Modified documents: 1

4.2 - New enrollment added to Alice
Modified documents: 1

4.3 - Bob's MA201 marks updated
Modified documents: 1

4.4 - PH203 enrollment removed from Diana
Modified documents: 1

4.5 - Upsert operation
New student inserted.
Inserted ID: 6a79a5764a48e250dcff6917
```

5. Write a Python script to delete a specific student by email and remove multiple students belonging to a specified major, printing the number of deleted documents.

```
from pymongo import MongoClient

client = MongoClient("mongodb://localhost:27017/")

db = client["university_mongo_assignment"]
students_collection = db["students_data"]

result = students_collection.delete_one(
    {
        "email": "newstudent@example.com"
    }
)

print("5.1 - Delete one student")
print("Number of documents deleted:", result.deleted_count)

result = students_collection.delete_many(
    {
        "major": "Arts"
    }
)

print("\n5.2 - Delete all Arts students")
print("Number of documents deleted:", result.deleted_count)

client.close()
```

Output:

```
In [6]: runfile('D:/TyBsc39/TY/untitled5.py', wdir='D:/TyBsc39/TY')
5.1 - Delete one student
Number of documents deleted: 1

5.2 - Delete all Arts students
Number of documents deleted: 2
```

6. Write a Python script to export Physics students to a JSON Lines file and import 2–3 new student documents from a JSON file into the students_data collection while handling duplicate or insertion errors.

```
from pymongo import MongoClient
import json

client = MongoClient("mongodb://localhost:27017/")

db = client["university_mongo_assignment"]
students_collection = db["students_data"]

students = students_collection.find(
    {
        "major": "Physics"
    }
)

with open(
    "physics_students.json",
    "w",
    encoding="utf-8"
) as file:

    for student in students:

        # Convert ObjectId to string
        student["_id"] = str(student["_id"])

        # Write JSON Lines format
        file.write(
            json.dumps(student) + "\n"
        )

print("6.1 - Physics students exported successfully!")
print("File: physics_students.json")

new_students = [
    {
        "name": "Jack Miller",
        "email": "jack.m@example.com",
        "major": "Physics",
        "enrollments": [
            {
                "course_code": "PH203",
                "title": "Quantum Mechanics",
                "marks": 87
            }
        ]
    },
    {
        "name": "Karen Lee",
        "email": "karen.l@example.com",
    }
```

```
"major": "Computer Science",
"enrollments": [
    {
        "course_code": "CS101",
        "title": "Intro to Programming",
        "marks": 95
    },
    {
        "course_code": "CS205",
        "title": "Database Management",
        "marks": 91
    }
]
},
{
    "name": "Leo Martin",
    "email": "leo.m@example.com",
    "major": "Arts",
    "enrollments": [
        {
            "course_code": "AR101",
            "title": "Art History",
            "marks": "Pending"
        }
    ]
}
]
```

```
with open(
    "new_students_for_import.json",
    "w",
    encoding="utf-8"
) as file:
```

```
    for student in new_students:
```

```
        file.write(
            json.dumps(student) + "\n"
        )
```

```
print("\n6.2 - new_students_for_import.json created successfully!")
```

```
students_to_insert = []
```

```
with open(
    "new_students_for_import.json",
    "r",
    encoding="utf-8"
) as file:
```

```
    for line in file:
```

```
if line.strip():

    student = json.loads(line)

    # Check if student already exists
    existing_student = students_collection.find_one(
        {
            "email": student["email"]
        }
    )

    if existing_student:

        print(
            "Student already exists. Skipping:",
            student["email"]
        )

    else:

        students_to_insert.append(student)

if students_to_insert:

    result = students_collection.insert_many(
        students_to_insert,
        ordered=False
    )

    print(
        "\n6.3 - Number of students imported:",
        len(result.inserted_ids)
    )

else:

    print("\n6.3 - No new students to import.")
    client.close()
    print("\nMongoDB connection closed.")
```

Output:

```
In [7]: runfile('D:/TyBsc39/TY/untitled6.py', wdir='D:/TyBsc39/TY')
6.1 - Physics students exported successfully!
File: physics_students.json

6.2 - new_students_for_import.json created successfully!

6.3 - Number of students imported: 3

MongoDB connection closed.
```